JAVASCRIPT



ALGORITHME

- Un algorithme est une manière de résoudre un problème.
- Un algorithme est une suite d'actions que devra effectuer un **automate** (ordinateur) pour arriver en un temps fini à un résultat déterminé.

Algorithme



Problème

Programme

Ecrire un algorithme qui saisit une date au clavier sous la forme : **Jour, Mois et Année** et l'affiche sous la forme **Jour/Mois/Année.** Exemple : 16/6/2020

```
ALGO Date
VAR
        Jour,Mois, Annee :entier
DEBUT
        Ecrire (" Donner le jour    :")
        Lire (Jour)
        Ecrire (" Donner le Mois    :")
        Lire (Mois)
        Ecrire (" Donner l'année    :")
        Lire (Annee)

        Ecrire (Jour,"/", Mois,"/", Annee)
FIN
```

Algorithme

```
<script>
    let Jour, Mois,Annee;

    Jour = prompt("Donner le jour");
   Mois = prompt("Donner le mois");
   Annee = prompt("Donner l'annee'");

    alert(Jour + "/" + Mois + "/" + Annee);
</script>
```

Programme en JavaScript

EXÉCUTION

- Une fois le programme écrit, on passe à l'exécution.
- On nomme **exécution** le fait de demander à un ordinateur de réaliser les ordres contenus dans un programme.
- Il faut savoir qu'un ordinateur ne sait manipuler que des informations écrites en chiffres binaires (0 et 1)

Quel que soit le langage avec lequel il est écrit, un programme doit être traduit en assembleur pour pouvoir être exécuté.

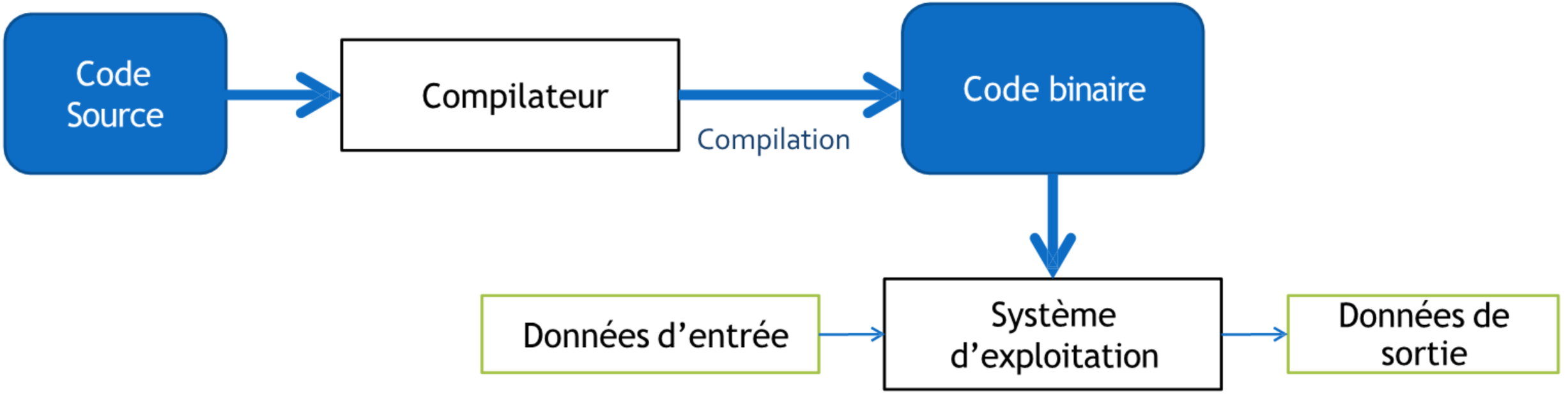
Ce processus de traduction dépend du langage choisi.

EXÉCUTION

compilé

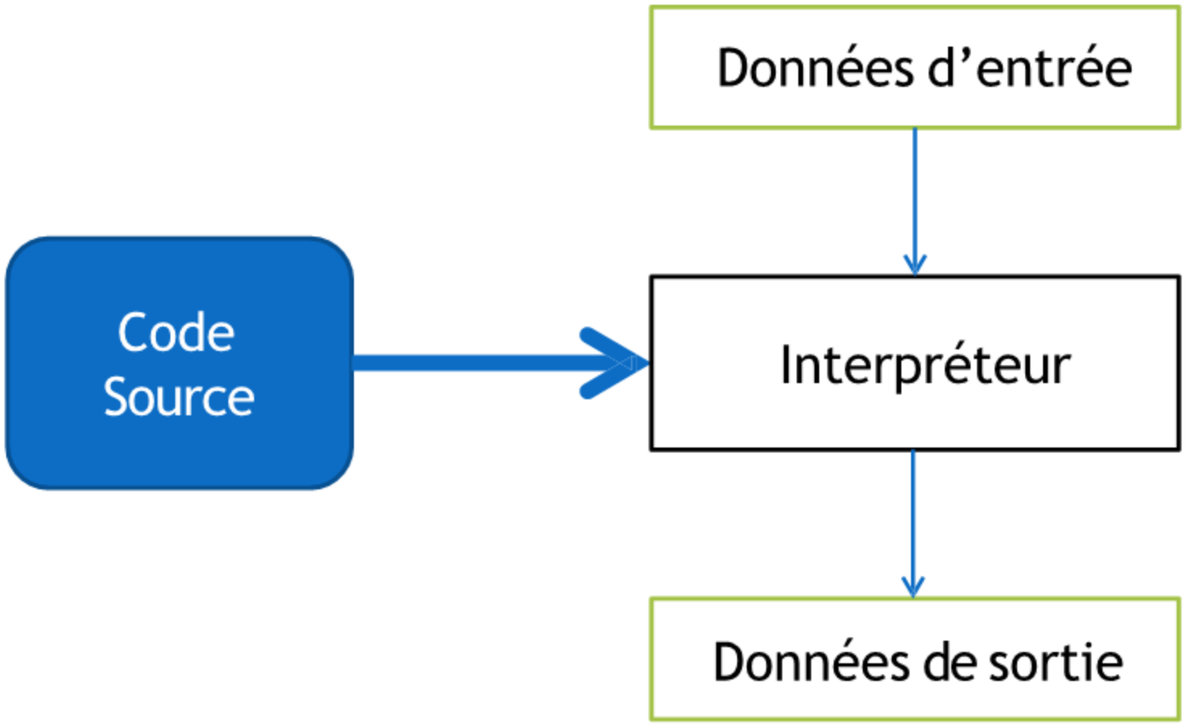
interprété

LE MODE COMPILATION



LE MODE INTERPRÉTATION

- Une fois le programme écrit, on passe à l'exécution



APPRENDRE À PROGRAMMER

- On ne crée pas un programme en se lançant directement dans l'écriture du code source.
- Il est d'abord nécessaire d'analyser le problème pour trouver la suite d'opérations à réaliser pour le résoudre.

STRUCTURE D'UN PROGRAMME

- Un programme informatique comme étant une liste d'ordres indiquant à un ordinateur ce qu'il doit faire.
- Ces ordres sont écrits sous forme de texte dans un ou plusieurs fichiers et forment ce qu'on appelle le **code source** du programme.
- Les lignes de texte dans un fichier de code source s'appellent des **lignes de code**.

INSTRUCTIONS

- Chaque ordre inclus dans un programme est appelée une **instruction**.
- Un programme est constitué d'une suite d'instructions.

Lorsqu'un programme est exécuté, les instructions qui le composent sont "lues" les unes après les autres.

Chaque instruction produit un résultat, et c'est la combinaison de ces résultats individuels qui produit le résultat final du programme.

EXPRESSION

- Une expression est une combinaison de **littéraux**, de **variables**, d'**opérateurs**, et de **fonctions** qui est évaluée (ou calculée) en suivant les règles de priorité et d'associativité du langage de programmation pour produire (ou retourner) une nouvelle valeur.

2+3	est une expression arithmétique qui vaut 5
y+5	est une expression
let d = c;	est une expression dont la valeur est celle de c
a && b	est une expression logique
let e = 3 + 2 * 4;	e contient 11 (3 + 8)
e = (3 + 2) * 4;	e contient 20 (5 * 4)

PRÉSENTATION

- Le Javascript est un langage de script orienté objet.
- Javascript permet de rendre dynamique un site internet développé en HTML.
- Javascript permet de développer de véritables applications fonctionnant exclusivement du côté client.
- Javascript a été initialement élaboré par ***Netscape*** en association avec ***Sun Microsystem***

LE STANDARD JAVASCRIPT

- «ES» est l'abréviation d'ECMAScript, le standard sur lequel repose JavaScript.
- Pendant longtemps, il s'agissait de l'ES5, la version 5 de l'ECMAScript.
- Depuis juin 2015, une nouvelle version est disponible : l'ES6.

Notation : ES6 = ES2015; ES7 = ES2016 ; etc.

JAVASCRIPT & JAVA

Javascript	Java
Langage interprété	Langage pseudo-compilé (chargement d'une machine virtuelle)
Code intégré au HTML	Code (applet) à part du document HTML, appelé à partir de la page
Langage peu typé	Langage fortement typé (déclaration du type de variable)
Liaisons dynamiques : les références des objets sont vérifiées au chargement	Liaisons statiques : les objets doivent exister au chargement (compilation)
Accessibilité du code	Confidentialité du code

HTML ET JAVASCRIPT

- Utilisation de la balise **<script>** avec un code en interne

```
<script  type="text/javascript">

</script>
```

- Utilisation de la balise **<script>** avec un code en externe

```
<script  type="text/javascript"    src="MonFichier.js"></script>
```

HTML ET JAVASCRIPT

- Le mode inline : en utilisant les événements

```
<button onMouseOver="alert('hover');" >Afficher</button>
```

```
<input     type="text" name="valeur"
           onFocus="this.style.backgroundColor = 'gray';
                    this.style.color = 'white'"
/>
```

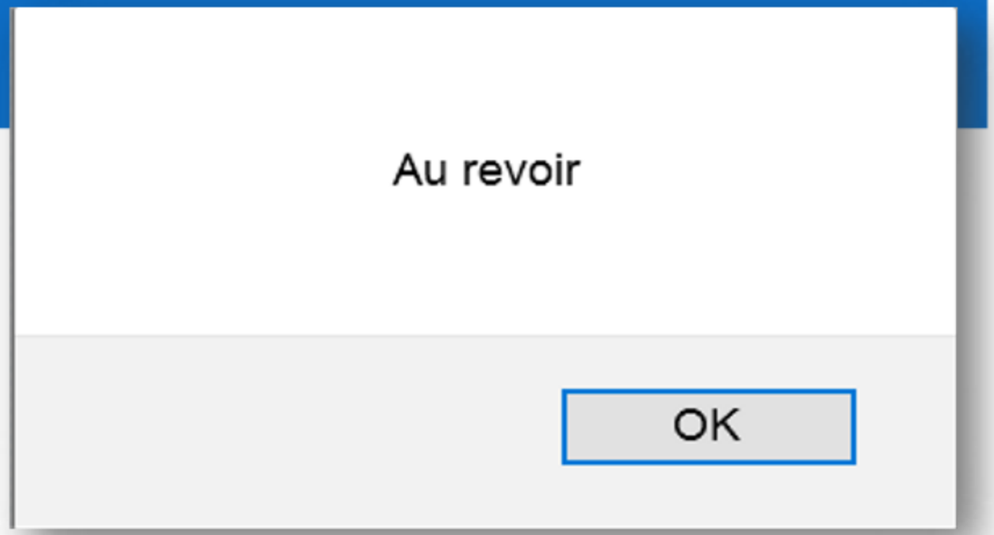
ENTRÉE ET SORTIE DE DONNÉES

- 3 types de boites de messages peuvent être affichés en utilisant Javascript
  - Méthode alert()
  - Méthode confirm()
  - Méthode prompt()
- La méthode **document.write** permet d 'écrire du code HTML dans la page WEB

ENTRÉE ET SORTIE DE DONNÉES

- Méthode alert()

```
alert("Au revoir");
alert(" Suite... ");
```



- Méthode prompt()

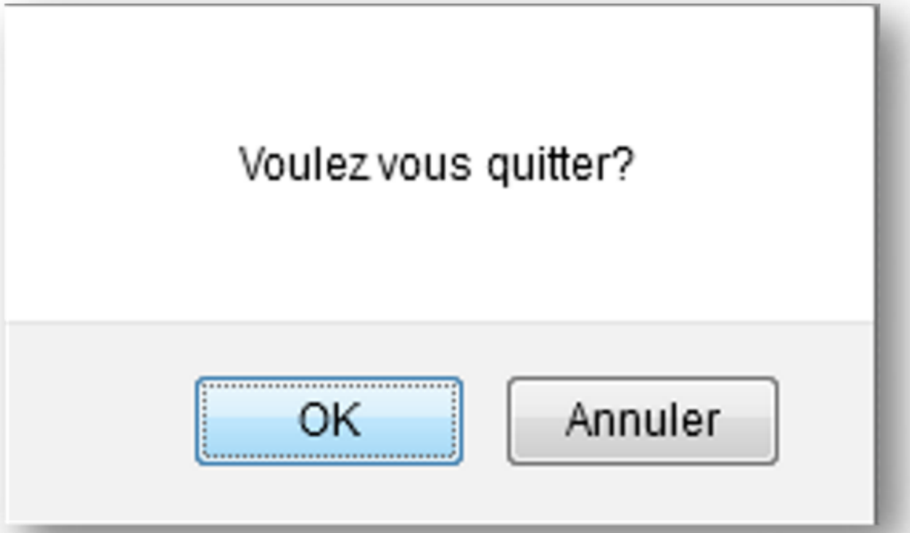
```
let nom = prompt("Votre nom?","Ali");
if (nom != null)
        alert("Bienvenue " + nom);
```



ENTRÉE ET SORTIE DE DONNÉES

- Méthode confirm()

```
if (confirm("Voulez vous quitter?"))
        alert("Au revoir");
else
        alert(" Suite… ");
```



- Méthode document.write()

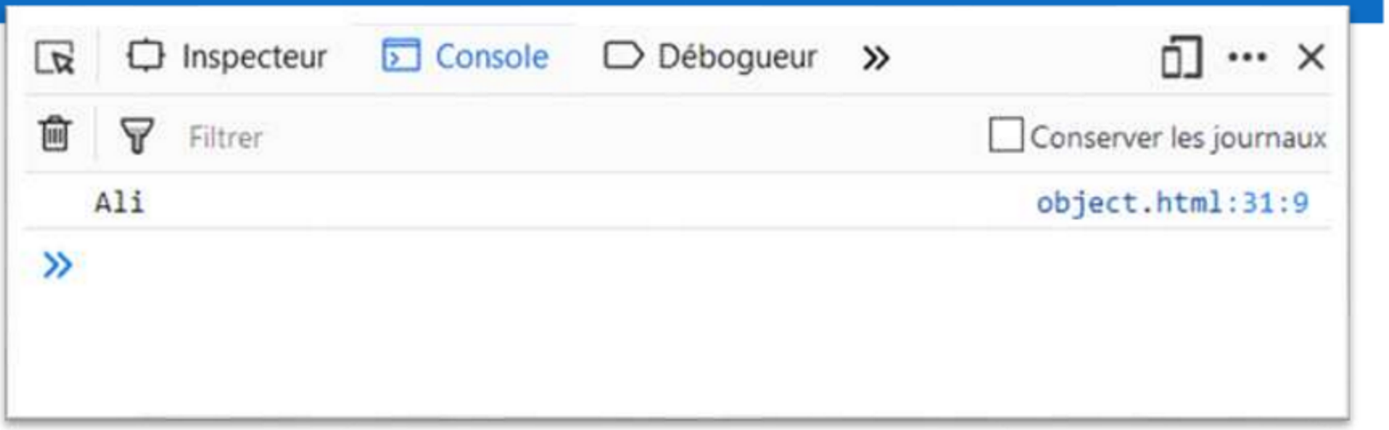
```
document.write("<h1>salut</h1>");
```



CONSOLE

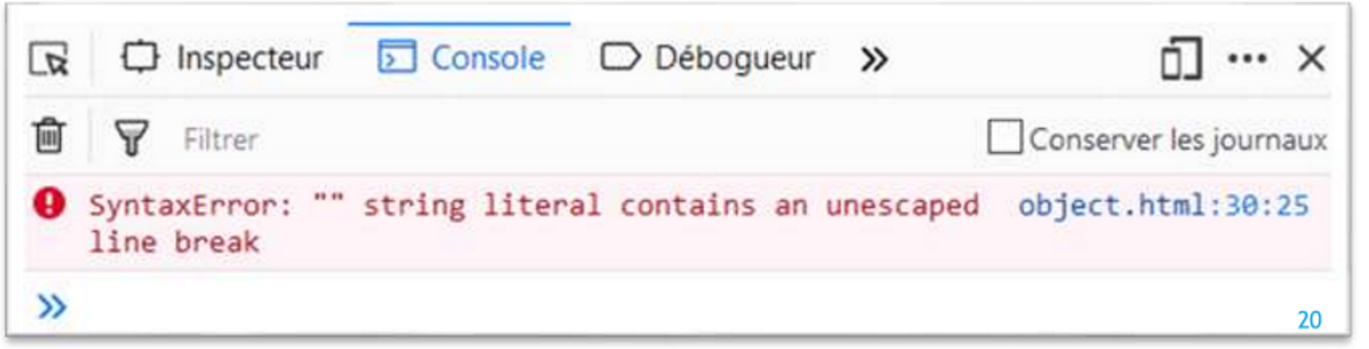
- Méthode console.log()

```
var nom = "Ali";
console.log(nom);
```



- Permet d'identifier les erreurs

```
var nom = "Ali';
console.log(nom);
```



SYNTAXE DE BASE

- Typologie
  - Toute instruction se termine par un point-virgule (**;**)
    - (**de préférence**)
  - Sensible à la casse

- Les commentaires
  - /* Voici un commentaire! */
  - // un commentaire sur une ligne

QU'EST-CE QU'UNE VARIABLE ?

- L'objectif d'un programme informatique est de faire quelque chose avec des données.
- Votre programme utilise des variables pour enregistrer et manipuler ces données.

Une variable est un **contenant** utilisé pour enregistrer une donnée spécifique dont votre programme a besoin pour travailler

- Une donnée placée dans une variable s'appelle une **valeur**.
- Votre programme utilise des variables pour enregistrer et manipuler ces données.

LES VARIABLES (1/5)

- Principe
  - Javascript est un langage interprété
  - Les variables n'ont pas besoin d'être déclarées
    - Typage implicite
  - La déclaration peut se faire d'une manière explicite grâce aux mots clefs ***let*** ou ***var***

LES VARIABLES (2/5)

- Les mot-clés ***var*** et ***let*** permettent de déclarer une ou plusieurs variables.

```
// Déclaration de i, de j et de k.
var i, j, k;
let i, j, k;
```

- Après la déclaration de la variable, il est possible de lui affecter une valeur par l'intermédiaire du signe d'égalité (=).

```
// Affectation de i.
i = 2;
```

- Si une valeur est affectée à une variable sans que cette dernière ne soit déclarée, alors JavaScript la déclare automatiquement.

```
// Déclaration et affectation de prix.
var prix = 0;
let prix = 0;
```

LES VARIABLES (3/5)

- Visibilité des variables
  - Variable locale
    - Visible uniquement à l'intérieur d'un contexte d'utilisation
    - Utilisation de **var**
  - Variable globale
    - Visible dans tout le script

LES VARIABLES (4/5)

- La lecture d'une variable non déclarée provoque une erreur
- Une variable correctement déclarée mais dont aucune valeur n'est affectée, est indéfinie (***undefined***).
- Les noms de variables ne peuvent contenir que des lettres, chiffres, ou le caractère "_" (***underscore***)
- Les caractères spéciaux et accentués sont interdits (é, à, ç, ï, etc..)
- Un nom de variable ne peut contenir d'espaces.
  - **Mon Prenom** n'est pas un nom de variable correct. Il y a un espace.
- Les mots réservés JavaScript ne peuvent être utilisés comme noms de variable.

LES VARIABLES (5/5)

- Notations :

```
var monPremierNom;          // Camel case

var MonPremierNom;          // Partial case

var mon_premier_nom;        // underscore
```

CONSTANTES

- certaines données ne seront pas modifiées pendant l'exécution du programme.
- Par exemple : Une date de naissance, PI, etc.

- Instruction ***const***
- L'instruction ***const*** a besoin d'être déclaré avec une valeur initiale.

```
const CONFIG = 'windows 10' ;      // déclaration d'une constante
CONFIG = 'Linux';                  // Retourne une erreur
const BASE ;                       // Retourne une erreur, il manque la valeur
```

LES OPÉRATEURS

- Les opérateurs
  - les opérateurs de calcul (+, -, /, %,*)
  - les opérateurs d'assignation ou affectation (=)
  - les opérateurs d'incrémentation (++, --, +=, etc)
  - les opérateurs de comparaison (>, <, <=, ==, ===, !=, !==, etc)
  - les opérateurs logiques (&&, ||, !)
  - Etc.

LA CONCATÉNATION

- L'opérateur + possède deux rôles:
  - Opérateur *arithmétique* : si tous les opérandes sont de type ***Number***
  - Opérateur de *concaténation* : si un des opérandes n'est pas de type ***Number***

```
firstName = 'Tom';
document.write("<p>Bonjour, je m'appelle " + firstName + '</p>');
```

QU'EST-CE QU'UN TYPE ?

- Le **type** d'une variable ou d'une constante est tout simplement le genre des données qu'elle enregistre.
- En JavaScript, il y a trois types primitifs principaux :
  - number (nombre) ;
  - string (chaîne de caractères) ;
  - boolean (valeur logique).

TYPES

- Types de bases
  - ***boolean***
  - ***number***
  - ***string***
- Type complexe
  - ***object***
- Types spéciaux
  - ***Null*** : possède une seule valeur, **nul**l, qui signifie l'absence de données dans une variable
  - ***undefined***si la valeur est indéfinie (variable déclarée mais pas initialisée ou variable non déclarée)
  - ***function*** si la valeur est une fonction

CONVERSIONS NUMÉRIQUES

- Relatifs aux valeurs et conversions des nombres, on trouve 4 fonctions :
  - ***isNaN*** détermine si un paramètre donné n'est pas un nombre
  - ***Number*** effectue une conversion
  - ***parseInt*** effectue une conversion en valeur entière
  - ***parseFloat*** effectue une conversion en valeur réelle

CONVERSIONS NUMÉRIQUES

- Exemples

```
alert(isNaN(10));                    // false
alert(isNaN("10"));                  // false - peut être convertie
alert(isNaN("blue"));                // true - ne peut être convertie
var num1 = Number("hello world");    // NaN
var num2 = Number("00001");          // 1
var num3 = Number(true);             // 1
var num3 = parseInt("");             // NaN
var num4 = parseInt(22.5);           // 22
var num5 = parseInt("70",10);        // 70 - la bsase 10 est spécifiée
var num6 = parseFloat("22.5");       // 22.5
```

LES OPÉRATIONS SUR LES CHAÎNES

- Exemples

```
var s = "hello world";
alert(s.length);                    // 11
alert(s.charAt(1));                 // "e"
alert(s.charCodeAt(1));             // 101
alert(s.slice(3));                  // "lo world"
alert(s.slice(-3));                 // "rld"
alert(s.substring(3,7));            // "lo w"
alert(s.indexOf("o"));              // 4
alert(s.lastIndexOf("o"));          // 7
alert(s.toUpperCase());             // HELLO WORLD
alert(s + " !");                    // hello world !
```

FONCTIONS MATHÉMATIQUES

- Exemples

```
alert(Math.E);                          // la valeur de e
alert(Math.PI);                         // la valeur de pi
alert(Math.min(5,12));                  // 5
alert(Math.max(23,5,7,130,12));         // 130
alert(Math.ceil(25.3));                 // 26
alert(Math.floor(25.8));                //25
alert(Math.random());                   // valeur aléatoire entre 0 et 1
```

Math.**abs(x)**	Math.**exp(x)**	Math.**log(x)**
Math.**pow(x,y)**	Math.**sqrt(x)**	
Math.**sin(x)**	Math.**cos(x)**	Math.**tan(x)**

FONCTION EVAL()

- Cette fonction exécute un code JavaScript à partir d'une chaîne de caractères

```
var ch = "5+2";
document.write(eval(ch));
```

```
7
```

MODE STRICT

- Le mode strict permet de détecter les problèmes dans votre code.
- Des exceptions sont alors émises.
- Si des actions potentiellement dangereuses, par exemple accéder à une variable globale, sont effectuées, le mode strict peut les prévenir ou bien émettre une erreur.
- Etc.

MODE STRICT

- Le mode strict peut être activé en écrivant simplement la chaîne de caractères ***use strict*** dans un fichier JavaScript.

```
1  "use strict";
2
3  // this code works the modern way
4  ...
```

EXERCICE 1 :CALCUL DE TVA

- Enoncé:

L'utilisateur saisit un montant HT, le résultat TTC s'affiche en HTML.

- Détails:

- Le taux de TVA utilisé est le taux normal en vigueur en France, il ne bouge pas.
- Il faut réafficher toutes les informations : le montant HT, le montant de TVA et le résultat TTC.
- Le montant HT est un nombre à virgule (on peut saisir des centimes).

LES INSTRUCTIONS CONDITIONNELLES (1/2)

- L'instruction if
  - if (condition réalisée) { liste d'instructions }
- L'instruction if … else
  - if (condition réalisée) {liste d'instructions}
    else { autre série d'instructions }
- Opérateur ternaire
  - (condition) ? instruction si vrai : instruction si faux

```
signe = (x >= 0)? 1 : -1;
```

LES INSTRUCTIONS CONDITIONNELLES (2/2)

- L'instruction switch

```
switch (Variable) {
    case Valeur1     : Liste d'instructions break;
    case Valeur1     : Liste d'instructions break;
    case Valeurs...  : Liste d'instructions break;
    default          : Liste d'instructions break;
}
```

EXERCICE 2 :CALCUL DE TVA AVEC REMISE

- Enoncé:

L'utilisateur saisit un montant HT et s'il le souhaite une remise, le résultat TTC s'affiche en HTML.

- Détails:

- L'utilisateur doit pouvoir répondre oui ou yes à la demande de remise.
- La saisie de la remise se fait en pourcentage, un nombre à virgule donc, et elle s'applique sur le montant HT autrement il y a fraude à la TVA ;-)
- Si une remise est appliquée, il faut à la fin réafficher le pourcentage de remise
- S'il n'y a pas de remise il faut à la fin afficher "Aucune remise n'a été appliquée"
- Il faut répéter le moins de code possible, notamment **__il ne faut pas répéter le calcul du montant TTC final__**.

EXERCICE 3 :CALCULATRICE NUMÉRIQUE

- Enoncé:

L'utilisateur saisit deux nombres ainsi que l'opération mathématique souhaitée, le résultat de cette opération s'affiche en HTML.

Les opérations mathématiques acceptées sont :

- L'addition : a + b
- La soustraction : a - b
- La multiplication : a * b
- La division : a / b
- La puissance : a ^ b (exemple : 3 ^ 2 = 9)

EXERCICE 3 :CALCULATRICE NUMÉRIQUE

- Détails:

- Il faut afficher une erreur en cas d'opération mathématique inconnue.
- Les nombres saisis peuvent être à virgule.
- L'utilisateur doit pouvoir autant saisir « le nom de l'opération » que « l'opérateur » correspondant : « + - * / ^ »
- Il faut répéter le moins de code possible, notamment le code d'affichage du résultat.

- Rappels:

- La division par zéro n'existe pas, il faut donc gérer ce cas…

EXERCICE 4 :CHIFOUMI

- Enoncé:

L'utilisateur saisit le mot « pierre », « feuille » ou « ciseau », l'ordinateur choisit aléatoirement l'une des trois possibilités, et la partie commence !

- Détails:

- Le mot peut tout aussi bien être saisi en minuscules qu'en majuscules.

- Rappels:

- Si le joueur et l'ordinateur font le même choix on obtient une égalité.
- Le ciseau est écrasé par la pierre (la pierre gagne, le ciseau perd).
- La feuille est découpée par le ciseau (le ciseau gagne, la feuille perd).
- La pierre est enveloppée par la feuille (la feuille gagne, la pierre perd).

LES INSTRUCTIONS ITÉRATIVES (1/3)

- La boucle for
  - for (expr1; expr2; expr3){instruction;}

```
for (let i=0; i<=10; i++) {
        document.write(i + "<br />");
}
```

```
for (let i=10; i>=0; i--) {
        document.write(i + "<br />");
}
```

LES INSTRUCTIONS ITÉRATIVES (2/3)

- La boucle while
  - While(condition) {bloc d'instructions ;}

```
i = 1;
while (i <= 10) {
        document.write(i + "<br />");
        i++;
}
```

LES INSTRUCTIONS ITÉRATIVES (3/3)

- La boucle do…while
  - do {bloc d'instructions ;} while(condition) ;

```
i = 1;
do {
        document.write(i + "<br />");
        i++;
} while ( !  (i > 10)  ) ;
```

EXERCICE 5 : LA PHRASE LA PLUS LONGUE

- Enoncé:

- Rechercher et afficher en HTML la chaîne ayant le plus grand nombre de caractères dans un tableau de chaînes.
- Construire le programme par étapes :
  1. faire une boucle qui affiche chaque phrase,
  2. faire une boucle qui affiche la longueur de chaque phrase,
  3. afficher un résultat (même s'il est faux),
  4. etc.

- Détails:

- Le programme doit à la fin afficher la phrase la plus longue « ET » la longueur de celle-ci.
- Le programme doit être dynamique : le tableau peut s'agrandir ou diminuer sans qu'il y ait besoin de modifier le code.

EXERCICE 6 : SAISIE OBLIGATOIRE

- Enoncé:

Demander à l'utilisateur de saisir obligatoirement un nombre quoiqu'il arrive, afficher ce nombre ensuite en HTML.

- Détails:

- On peut saisir un nombre entier comme un nombre à virgule.

- Rappels:

- Il existe un moyen déjà vu précédemment pour vérifier qu'une valeur et un nombre ou pas…

EXERCICE 7 : TABLE DE MULTIPLICATION

- Enoncé:

- Construire une table des multiplications en JavaScript puis l'afficher en HTML.

- Détails:

- Demander à l'utilisateur de saisir la taille de la table des multiplications (exemple : si on saisit 10 il faut faire une table de 1 à 10).
- Il faut utiliser les balises HTML de tableaux pour construire l'affichage.
- Pour l'affichage, lorsque le nombre est multiplié par lui-même (1x1, 2x2, 3x3, etc.), la cellule du tableau HTML doit s'afficher d'une autre couleur que les autres cellules. **La solution doit être en CSS**.

LES FONCTIONS

- Déclaration et appel d'une fonction

```
function nom_fonction(arg1, arg2, ...argn)
{
    déclaration des variables ;
    bloc d'instructions ;
    return resultat ;       // Si la fonction retourne un résultat
}
```

PORTÉE DES VARIABLES

- Selon l'endroit où une variable est déclarée, celle-ci pourra être accessible de tout endroit dans le script ou bien uniquement dans une portion du code, on parle de « **portée** » d'une variable.
- En Javascript, les variables peuvent être ***globales*** ou ***locales***.
  - Une variable **globale** est déclarée en début de script et on peut l'atteindre de n'importe quel endroit du scripte.
  - Une variable locale est déclarée à l'intérieur d'une fonction ou d'un bloc et n'est utilisable que dans la fonction ou le bloc lui même.

PORTÉE DES VARIABLES

```
var x = 1;
if (x == 1)
{
        var x = 2;
        console.log(x);     // retourne 2
}
      console.log(x);       // retourne 2
```

```
let x = 1;
if (x == 1)
{
        let x = 2;
        console.log(x);     // retourne 2
}
      console.log(x);       // retourne 1
```

PORTÉE DES VARIABLES

```
for (var i = 0; i < 4; i++) {
var j = 10;
}
console.log(i);     // retourne 4
console.log(j);     // retourne 10
```

```
for (let i = 0; i < 4; i++) {
let j = 10;
}
console.log(i);     // Erreur
console.log(j);     // Erreur
```

LES TABLEAUX (1/5)

- Création à l'aide de l'objet **Array()**
- Les éléments d'un tableau peuvent appartenir à des types distincts
- L'index d'un tableau en Javascript commence de **0**

```
var colors = new Array();                          // tableau vide
var colors2 = new Array(20);                       // tableau avec 20 cases
var colors3 = new Array("red","blue","green");     // 3 cases
var colors4 = ["red","blue","green"];              // notation littérale
```

LES TABLEAUX (2/5)

- Exemples

```
var colors = ["red","blue","green"];
alert(colors.length);                    // 3
colors[colors.length]="black";           // nouvelle couleur
colors[99]="pink";
alert(colors.length);                    // 100
alert(colors[50]):                       // undefined
colors.length=10;                        // plus que 10 cases
```

LES TABLEAUX (3/5)

- Exemples

```
var colors = ["red","blue","green"];
alert(colors);                          // red,blue,green
alert(colors.join(";"));                // red;blue;green
colors.push("black");                   //Ajouter un élément à la fin du tableau
alert(colors);                          // red,blue,green,black
var item = colors.pop();                // Enlever le dernier élément du tableau
alert(item + " " + colors);             // black red,blue,green
var item2= colors.shift();              // Enlever le premier élément du tableau
alert(item2 + " " + colors);            // red blue,green
colors.unshift("yellow");               // Ajouter un élément au début du tableau
alert(colors);                          // yellow,blue,green
```

LES TABLEAUX (4/5)

- Exemples

```
var values = [0, 1, 2, 3, 4];
alert(values.reverse()); // 4,3,2,1,0
values = [0, 1, 15, 10, 5];
alert(values.sort()); // 0, 1, 15, 10, 5
alert(values.sort(function(a, b){return a-b})); // 0,1,5,10,15     si a-b>=0
alert(values.sort(function(a, b){return a-b})); // 15,10,5,1,0     si a-b<=0
```

LES TABLEAUX (5/5) : TRAVAIL À FAIRE

- Préparer un exemple pour chacune des fonctions suivantes :
  - splice
  - slice
  - filter
  - delete

LES DATES (1/3)

- L'objet Date est utilisé pour manipuler la date et l'heure
- Pour créer un objet date avec la date et l'heure ***actuelle*** on utilise l'instruction **new Date()**

```
var d = new Date();          // Thu Jun 18 2020 11:15:34 GMT+0100 (Central European Standard Time)
var d = new Date(milliseconds);      // var d = new Date(1634875872656);
var d = new Date(dateString);        // var d = new Date('December 17, 2005 03:24:00');
var d = new Date(year, month, day, hours, minutes, seconds, milliseconds);
// var d = new Date(2005, 11, 17, 03, 24, 00); (mois indexé par 0)
```

LES DATES (2/3)

- En JavaScript, lors de l'affichage, les dates sont automatiquement converties en format texte avec la méthode *toString()*
- La méthode *toUTCString()* permet de convertir une date au format d'affichage standard
- La méthode *toDateString()* permet de convertir une date en un format plus lisible

```
var d1 = new Date();          // Thu Jun 18 2020 11:15:34 GMT+0100 (Central European Standard Time)
var d2  = d1.toUTCString();       //Thu, 18 Jun 2020 11:15:34
var d3 = d1.toDateString();       //  Thu, 18 Jun 2020
```

LES DATES (3/3)

- Exemples de méthodes prédéfinies

getDate()	retourne le jour du mois (1-31)
getDay()	retourne le jour de la semaine (0-6)
getMonth()	retourne le mois (0-11)
getFullYear()	retourne l'année (YYYY)
getHours()	retourne l'heure (0-23)
getMinutes()	retourne les minutes (0-59)
getSecondes()	retourne les secondes (0-59)

EXERCICE 8 :DATE DU JOUR

- Enoncé:

Afficher dynamiquement la date du jour en HTML sous la forme « Nous sommes le Mardi 15 Septembre 2020 »

- Détails:

- Il va falloir se servir de tableaux pour afficher en français les noms des jours de la semaine et des mois…

LES OBJETS (1/3)

- Utilisé pour stocker des données
- Création à l'aide de l'objet **Object()**

```
var personne = new Object();
personne.nom = "Wissam";
personne.age = 23;
        //ou bien
var personne = { nom : "Wissam",  age : 23  }
```

LES OBJETS (2/3)

- Il est possible d'utiliser les crochets pour accéder à un champ

```
alert(personne.nom);
alert(personne["nom"]);
```

- On peut utiliser une boucle spéciale pour parcourir un objet

```
for (var item in personne)
        document.write(item + " : " + personne[item] );
//nom : Wissam age : 23
```

LES OBJETS (3/3)

- Un objet peut contenir une fonction appelée *méthode*

```
var person = {
        nom : "Abidi",
        prenom : "Nizar",
        id : 2222,
        nomComplet : function () {
                return this.nom + " " + this.prenom;
        }
};
```

```
document.write(person.nomComplet());
```

EXERCICE 9 : OBJET VOITURE

- Enoncé:

- Créer puis afficher en HTML un objet représentant une voiture :
  - La marque de la voiture
  - Son année de fabrication
  - Sa date d'achat
  - La liste des passagers (au moins 2), avec le prénom de chacun des passagers

- Détails:

- Afficher les propriétés de l'objet voiture sous la forme d'une liste HTML

LES ÉVÈNEMENTS

- Les évènements sont des actions de l'utilisateur, qui vont pouvoir donner lieu à une interactivité.

```
onEvenement="Action_Javascript_ou_Fonction();"
```

- Ce sont les gestionnaires d'événements qui permettent d'associer une action à un événement

```
<input type="button" onClick="Afficher()" value="Afficher" />
```

Evénement	Description
onClick	Lorsque l'utilisateur clique sur un bouton, un lien ou tout autre élément.
onDbClick	Lorsque l'utilisateur double clique sur un bouton, un lien ou tout autre élément.
onLoad	Lorsque la page est chargée par le browser ou le navigateur.
onUnload	Lorsque l'utilisateur quitte la page.
onMouseOver	Lorsque l'utilisateur place le pointeur de la souris sur un lien ou tout autre élément.
ondrag	Lorsque l'utilisateur effectue un glisser sur la fenêtre du navigateur.
ondrop	Lorsque l'utilisateur effectue un déposer sur la fenêtre du navigateur.
onMouseOut	Lorsque le pointeur de la souris quitte un lien ou tout autre élément.
onMouseUp	Lorsque l'utilisateur relâche le bouton de la souris
onMouseDown	Lorsque l'utilisateur appuie sur le bouton de la souris sur un élément
onFocus	Lorsqu' un élément de formulaire a le focus c.-à-d. devient la zone d'entrée active.

Evénement	Description
onBlur	Lorsqu'un élément de formulaire perd le focus c-à-d que l'utilisateur clique hors du champ et que la zone d'entrée n'est plus active.
onChange	Lorsque la valeur d'un champ de formulaire est modifiée.
onSelect	Lorsque l'utilisateur sélectionne un champ dans un élément de formulaire.
onSubmit	Lorsque l'utilisateur clique sur le bouton Submit pour envoyer un formulaire
onError	Lorsqu'une erreur apparaît durant le chargement de la page.
onKeyDown	Lorsque l'utilisateur appuie sur une touche de son clavier.
onKeyPress	Lorsque l'utilisateur maintient une touche de son clavier enfoncée.
onKeyUp	Lorsque l'utilisateur relâche une touche de son clavier préalablement enfoncée.
onReset	Lorsque l'utilisateur efface les données d'un formulaire à l'aide du bouton Reset.
onResize	Lorsque l'utilisateur redimensionne la fenêtre du navigateur
onAbort	Lorsque le chargement d'une ressource a été annulé
...	